

LD8-S0 Periodic Kernel Block-Routing Specification

Immutable source-independent routing, exact bit layout, terminal blocks, and cache ownership

mdstats development specification

2026-07-21

Contents

1	LD8-S0 - Periodic kernel block routing	1
1.1	Purpose	1
1.2	Public records	1
1.3	Input and output functions	2
1.4	Canonical signed-offset convention	2
1.5	Normative local bit layout	2
1.6	Relative block grouping	3
1.7	Terminal partial blocks	3
1.8	Cache contract	3
1.9	Constraints and failure behavior	4
1.10	Determinism	4
1.11	Focused validation	4
1.12	External attribution	4

1 LD8-S0 - Periodic kernel block routing

Package target: mdstats 0.19.55a0

Status: implemented

Primary module: mdstats.plotting.density_block_routing

1.1 Purpose

LD8-S0 separates reusable Gaussian-stencil geometry from field-specific source and target support. The routing record contains no atom positions, source nodes, source masks, or target blocks. It may therefore be reused only when its exact immutable stencil and storage layout match.

This stage changes storage and planning contracts only. It does not change the CIC estimator, Gaussian stencil, density values, normalization, HDR semantics, or renderer.

1.2 Public records

```
@dataclass(frozen=True, slots=True)
class BlockOffsetStencilGroup:
    nominal_block_offset: tuple[int, int, int]
    stencil_indices: NDArray[np.int32]
    signed_offsets: NDArray[np.int32]
    local_remainders: NDArray[np.int16]

@dataclass(frozen=True, slots=True)
class PeriodicKernelBlockRouting:
    logical_grid_shape: tuple[int, int, int]
    storage_block_shape: tuple[int, int, int]
```

```

block_grid_shape: tuple[int, int, int]
axis_block_extents: tuple[tuple[int, ...], ...]
stencil_identity: str
signed_offsets: NDArray[np.int32]
relative_block_offsets: NDArray[np.int32]
grouped_stencil_ranges: tuple[BlockOffsetStencilGroup, ...]
terminal_extent_classes: NDArray[np.int32]
terminal_validity_bitsets: NDArray[np.uint64]
metadata: FrozenJSONMapping

```

The schema identifiers are:

```

mdstats.block-offset-stencil-group.v1
mdstats.periodic-kernel-block-routing.v1
mdstats.density-routing-cache-info.v1

```

1.3 Input and output functions

```

canonical_signed_stencil_offsets(
    stencil: PeriodicGaussianStencilSupport,
) -> NDArray[np.int64]

build_periodic_kernel_block_routing(
    stencil: PeriodicGaussianStencilSupport,
    *,
    storage_block_shape: tuple[int, int, int] = (16, 16, 16),
    max_stencil_offsets: int = 2_000_000,
) -> PeriodicKernelBlockRouting

get_periodic_kernel_block_routing(
    stencil: PeriodicGaussianStencilSupport,
    *,
    storage_block_shape: tuple[int, int, int] = (16, 16, 16),
    max_stencil_offsets: int = 2_000_000,
    use_cache: bool = True,
) -> tuple[PeriodicKernelBlockRouting, bool]

```

Cache management is explicit:

```

clear_density_routing_cache() -> None
density_routing_cache_info() -> DensityRoutingCacheInfo

```

1.4 Canonical signed-offset convention

The canonical stencil stores modular logical indices. Routing converts each axis coordinate $q \in [0, N)$ to one signed representative:

$$\delta(q) = \begin{cases} q, & q \leq \lfloor N/2 \rfloor, \\ q - N, & q > \lfloor N/2 \rfloor. \end{cases}$$

For an even logical size, the Nyquist coordinate is represented positively. The conversion is deterministic and preserves the exact modular action

$$(\mathbf{n} + \delta) \bmod \mathbf{N}.$$

1.5 Normative local bit layout

For a storage block of shape (B_x, B_y, B_z) , local coordinates use C-order flattening:

$$i = (xB_y + y)B_z + z.$$

Bit i is stored in:

$$\text{word} = \left\lfloor \frac{i}{64} \right\rfloor, \quad \text{bit} = i \bmod 64.$$

The bit is the little-endian bit within its `uint64` word. For the default 16^3 block, one mask contains 4096 bits or 64 words, requiring 512 bytes.

The public helpers are:

```
pack_local_indices(...)
unpack_local_bitset(...)
bitset_popcount(...)
validity_bitset_for_extent(...)
```

1.6 Relative block grouping

For signed stencil offset δ and storage-block vector $\mathbf{B}$:

$$\mathbf{b}_\delta = \left\lfloor \frac{\delta}{\mathbf{B}} \right\rfloor, \quad \mathbf{r}_\delta = \delta - \mathbf{b}_\delta \mathbf{B}.$$

Offsets sharing $\mathbf{b}_\delta$ are grouped deterministically. The groups compactly describe the possible relative block displacements, but they do not store a fine routing table proportional to

$$B_x B_y B_z |\mathcal{K}_\varepsilon|.$$

Such a table is explicitly forbidden.

1.7 Terminal partial blocks

For logical size N_a and block size B_a , the block-lattice size is

$$G_a = \left\lceil \frac{N_a}{B_a} \right\rceil.$$

All nonterminal blocks have extent B_a . The terminal extent is

$$E_a = N_a - (G_a - 1)B_a.$$

Each axis records the exact extent of every block. The Cartesian product of unique axis extents produces at most eight three-dimensional extent classes. Every extent class has a validity bitset; invalid terminal slots can never enter source or target support.

1.8 Cache contract

The routing cache key includes:

- the exact SHA-256 identity of the normalized finite stencil;
- logical grid shape and terminal layout through that stencil identity;
- storage-block shape.

The stencil identity includes exact float64 cell bytes, Gaussian-bandwidth bits, tail-tolerance bits, active modular indices, and normalized weights.

The cache is process-local, thread-safe, byte-bounded, clearable, and inspection-capable. It stores only source-independent routing. A field-specific packed source or support atlas is never entered into this cache.

1.9 Constraints and failure behavior

Construction fails before retained routing allocation when:

- any shape is not a positive three-tuple;
- the stencil schema is unsupported;
- active stencil indices are invalid or noncanonical;
- the stencil-offset count exceeds `max_stencil_offsets`;
- terminal extent records are inconsistent;
- bitsets contain invalid local positions.

Caller limits are rechecked on cache hits.

1.10 Determinism

The following order is normative:

1. stencil offsets follow the canonical stencil's increasing modular flat index;
2. relative block groups are sorted lexicographically;
3. stencil indices within each group preserve canonical stencil order;
4. terminal extent classes are sorted lexicographically;
5. JSON arrays preserve the same order.

Repeated construction from identical inputs is byte-identical on the validation platform.

1.11 Focused validation

Tests cover:

- local bit packing and unpacking;
- terminal validity for divisible and partial grids;
- positive-Nyquist signed-offset convention;
- grouped routing identity and deterministic order;
- JSON round trips;
- cache hit, miss, clear, byte accounting, and caller-limit revalidation;
- rejection of invalid shapes, offsets, and terminal masks.

1.12 External attribution

The finite stencil is the already specified periodized Gaussian operator. CIC deposition follows Hockney and Eastwood, *Computer Simulation Using Particles* (1988). The source-independent terminal-aware block-routing and normative bit layout are mdstats-specific designs.